

# You have Done !

- Internal Sorting Algorithms
  - Uses Primary memory for sorting or program stack .
- External Sorting Algorithms
  - Uses Secondary memory /hard disk
- In-place Sorting Algorithms
  - No need extra memory area
- Stable Sorting Algorithms
  - key or value remains in relative position after sort
- Bubble Sort algo and complexity
- Selection Sort algo and complexity
- Insertion Sort algo and complexity

# Divide and Conquer

**“Divide and Conquer” is an algorithm design paradigm.**

A divide-and-conquer algorithm recursively breaks down a problem into two or more sub-problems of the same or related type, until these become simple enough to be solved directly.

The solutions to the sub-problems are then combined to give a solution to the original problem.

# Divide and Conquer

Greedy Method

Dynamic Programming

Linear Programming

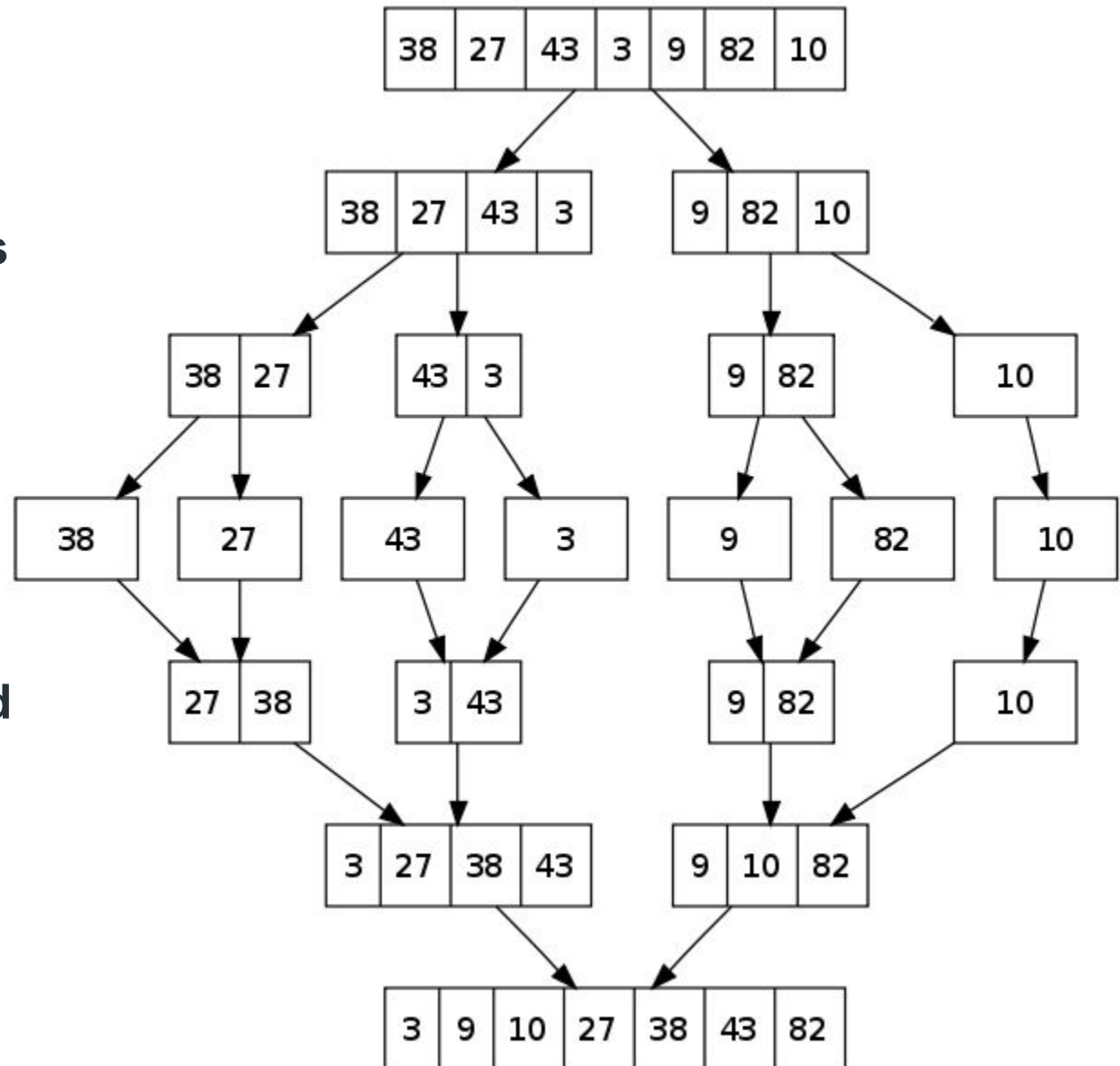
Randomized Algorithms

Approximation so on

Recursion is Used  
almost in all paradigms

Recall How program

**Recursion Stack** is used



# Binary Search

**BinarySearch**(A , l, r, key)

```
{
    if(l == r)
    {
        if(key == A[l])
            return l+1; //index starts from 0
        else
            return 0;
    }

    else
    {
        m = (l + r) / 2 ; //integer division
        if(key == A[m])
            return m+1;
        else if (key < A[m])
            return BinarySearch(l, m-1, key) ;
        else return BinarySearch(m+1, r, key) ;
    }
}
```

Running time of the algorithm is:

$$T(n) = T(n/2) + O(1) \\ = O(\log n) .$$

In the best case output is obtained at one run i.e. **O(1)** time if the key is at middle.

In the worst case the output is at the end of the array so running time is **O(logn)** time.

In the average case also running time is **O(logn)**. For unsuccessful search best, worst and average time complexity is **O(logn)**.

# Min-Max algorithm

```

MinMax(l,r)
{
    if(l == r)
        max = min = A[l];
    else if(l = r-1)
    {
        if(A[l] < A[r])
        {
            max = A[r]; min = A[l];
        }
        else
        {
            max = A[l]; min = A[r];
        }
    }
    else
    {
        //Divide the problems
        mid = (l + r)/2; //integer division
        //solve the subproblems
        {min,max}=MinMax(l,mid);
        {min1,max1}= MinMax(mid +1,r);
        //Combine the solutions
        if(max1 > max) max = max1;
        if(min1 < min) min = min1;
    }
}

```

## Analysis:

We can give recurrence relation as below for MinMax algorithm in terms of number of comparisons.

$$T(n) = 2T(n/2) + 1, \text{ if } n > 2$$

$$T(n) = 1, \text{ if } n \leq 2$$

Solving the recurrence by using master method complexity is (case 1)  $O(n)$ .

# Merge Sort

The *merge sort* algorithm closely follows the divide-and-conquer paradigm. Here we deal with the sub problems, we state each sub problem as sorting a sub array  $A[p..r]$ . Initially,  $p = 1$  and  $r = n$  but these values change as we recurse through sub problems.

To sort  $A[p..r]$ :

**Divide** by splitting into two subarrays  $A[p..q]$  and  $A[q + 1..r]$ , where  $q$  is the halfway point of  $A[p..r]$ .

**Conquer** by recursively sorting the two subarrays  $A[p..q]$  and  $A[q + 1..r]$ .

**Combine** by merging the two sorted subarrays  $A[p..q]$  and  $A[q + 1..r]$  to produce a single sorted subarray  $A[p..r]$ , and

the procedure **MERGE**( $A, p, q, r$ ), to accomplish this step,

**The recursion bottoms out when the sub array has just 1 element, so that it's trivially sorted.**

**MERGE-SORT**(A, p, r)

if  $p < r$        $\backslash\backslash$  Check for base case

**then**  $q \leftarrow (p + r)/2$      $\backslash\backslash$  Divide

**MERGE-SORT**(A, p, q)  $\backslash\backslash$  Conquer

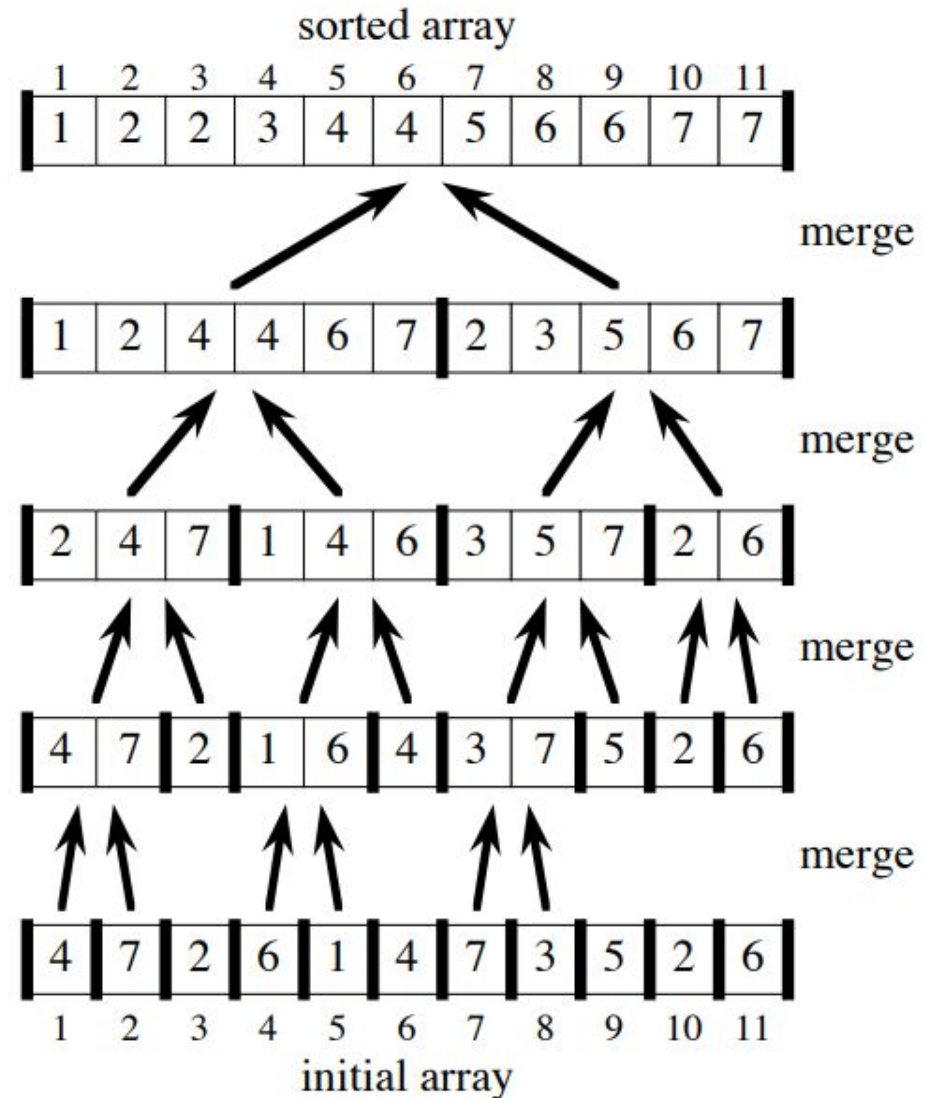
**MERGE-SORT**(A, q + 1, r)  $\backslash\backslash$  Conquer

**MERGE**(A, p, q, r)       $\backslash\backslash$  Combine

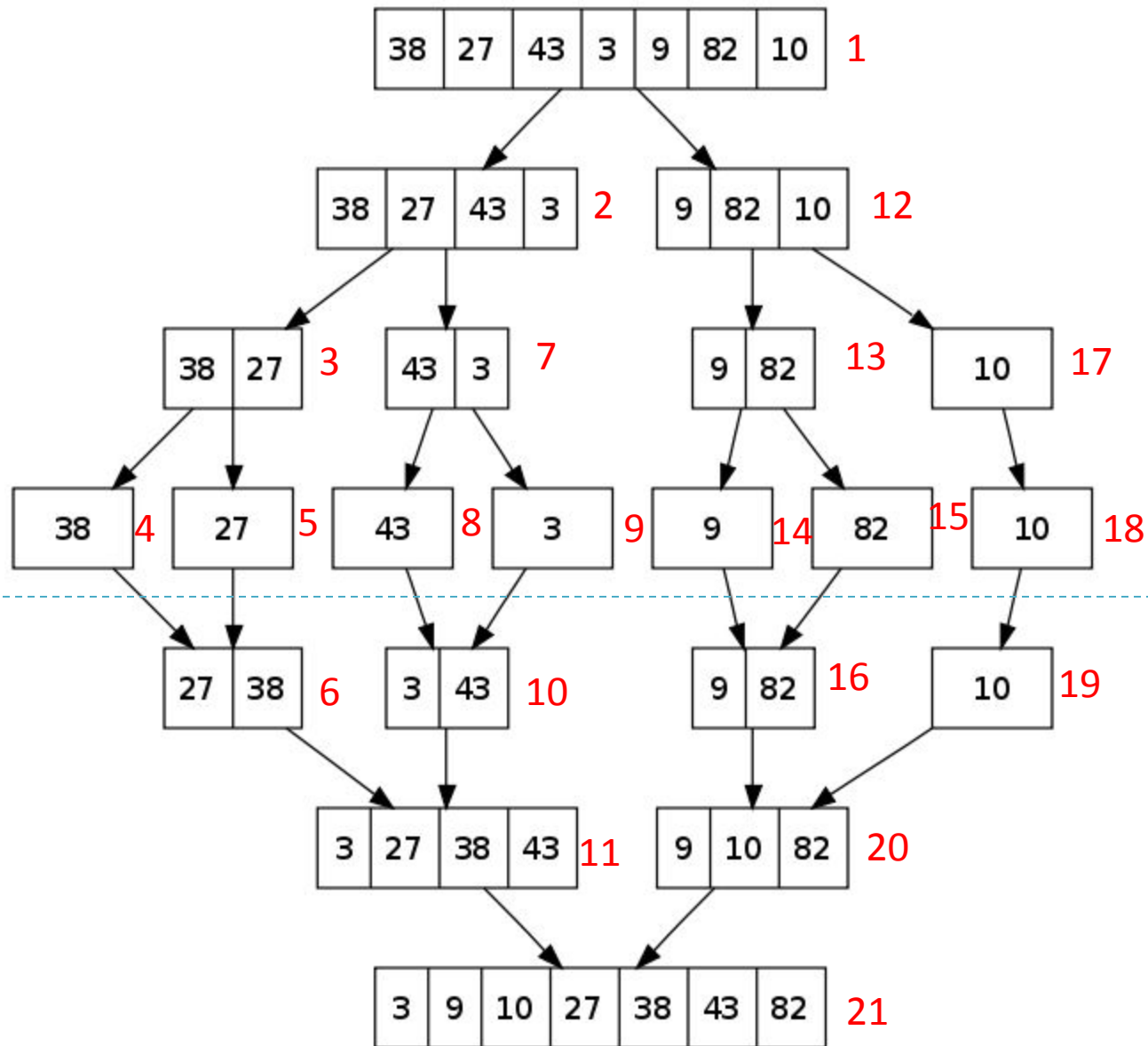
**Initial call:** **MERGE-SORT**(A, 1, n)

**Example:** Bottom-up view for  $n = 11$

(Heavy lines shows subarrays used in subproblems)



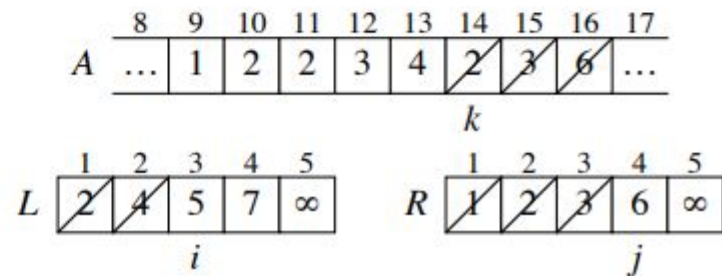
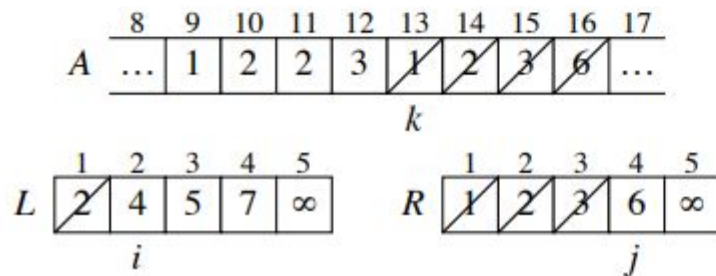
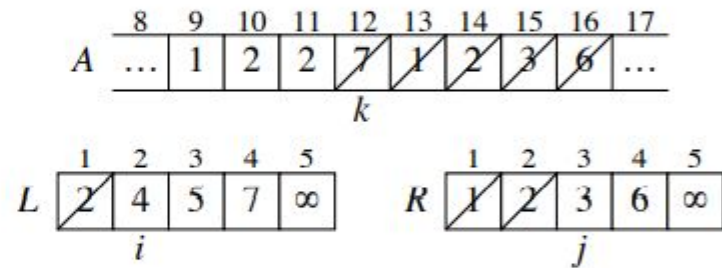
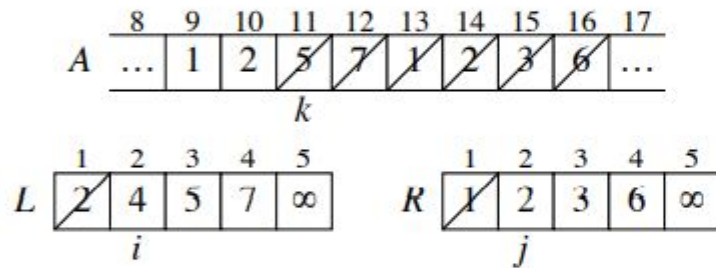
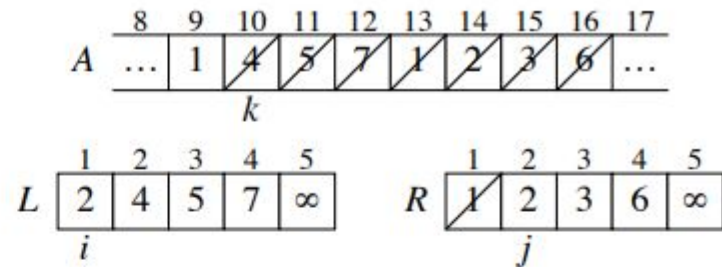
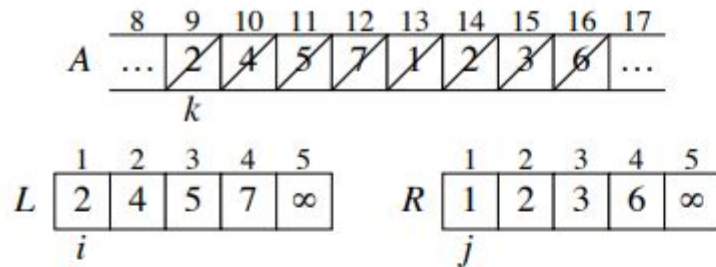
# How recursion stack works ?

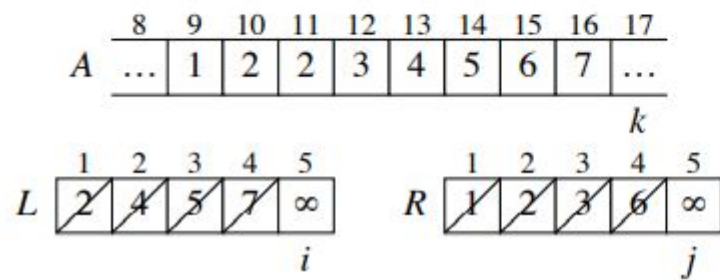
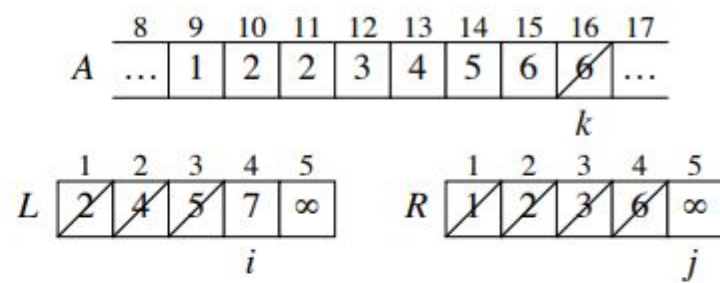
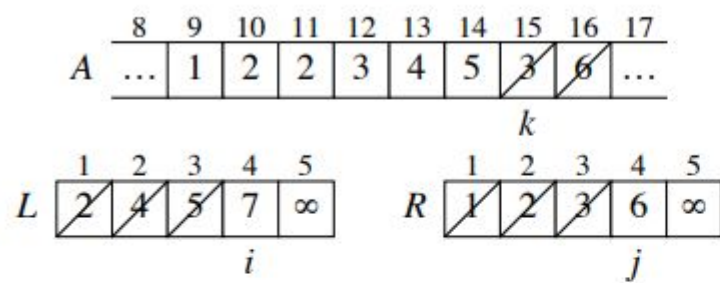




**Merge Procedure:****MERGE**( $A, p, q, r$ ) $n1 \leftarrow q - p + 1$  $n2 \leftarrow r - q$ create arrays  $L[1 \dots n1 + 1]$  and  $R[1 \dots n2 + 1]$ **for**  $i \leftarrow 1$  **to**  $n1$ **do**  $L[i] \leftarrow A[p + i - 1]$ **for**  $j \leftarrow 1$  **to**  $n2$ **do**  $R[j] \leftarrow A[q + j]$  $L[n1 + 1] \leftarrow \infty$  $R[n2 + 1] \leftarrow \infty$  $i \leftarrow 1$  $j \leftarrow 1$ **for**  $k \leftarrow p$  **to**  $r$ **do if**  $L[i] \leq R[j]$ **then**  $A[k] \leftarrow L[i]$  $i \leftarrow i + 1$ **else**  $A[k] \leftarrow R[j]$  $j \leftarrow j + 1$

# Example: A call of MERGE(9, 12, 16)





# Analyzing merge sort

## Time Complexity

Each divide step yields two sub problems, both of size exactly  $n/2$ .

The base case occurs when  $n = 1$ .

When  $n \geq 2$ , time for merge sort steps:

**Divide:** Just compute  $q$  as the average of  $p$  and  $r \Rightarrow \Theta(1)$ .

**Conquer:** Recursively solve 2 sub problems, each of size  $n/2 \Rightarrow 2T(n/2)$ .

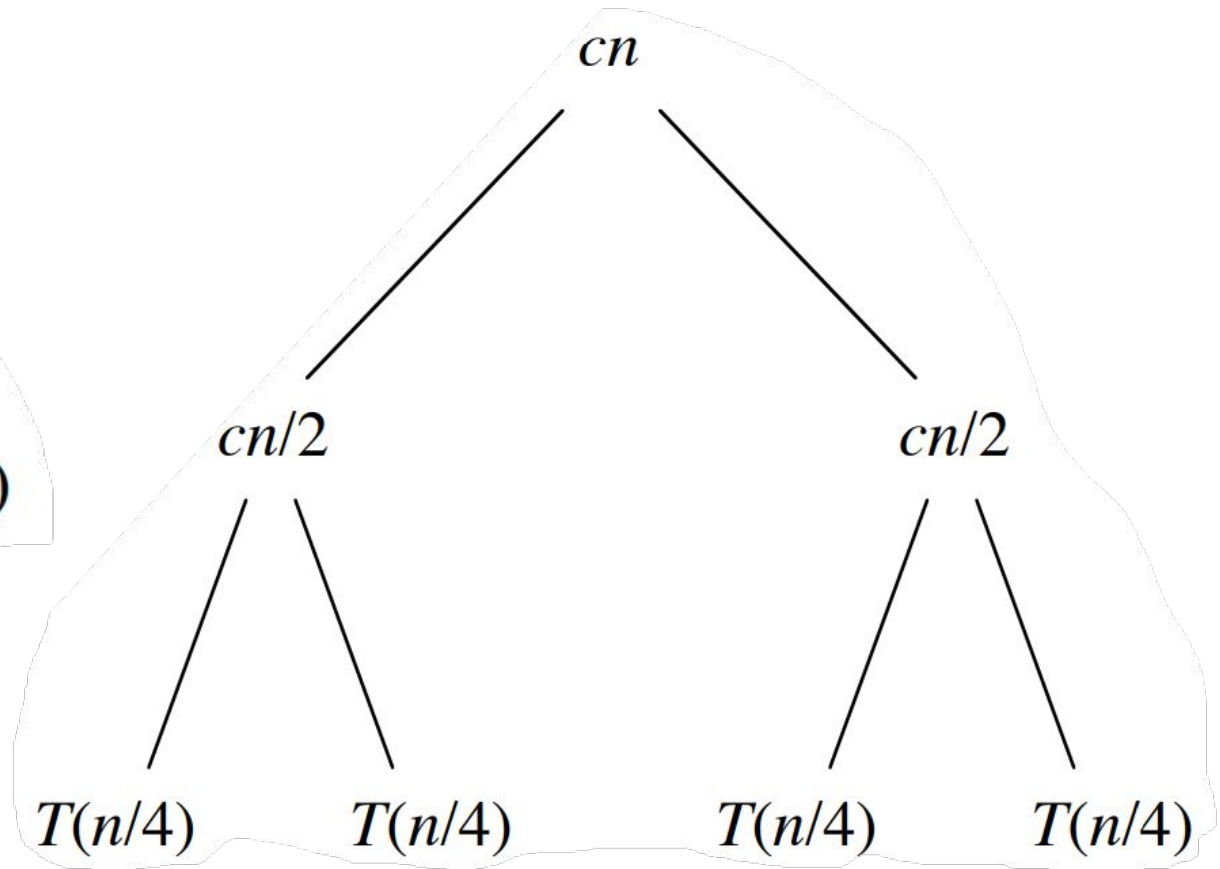
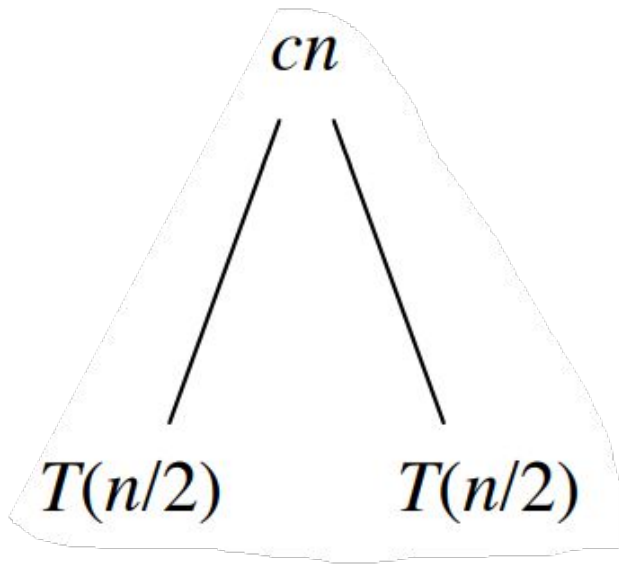
**Combine:** MERGE on an  $n$ -element sub array  $\Rightarrow \Theta(n)$  time.

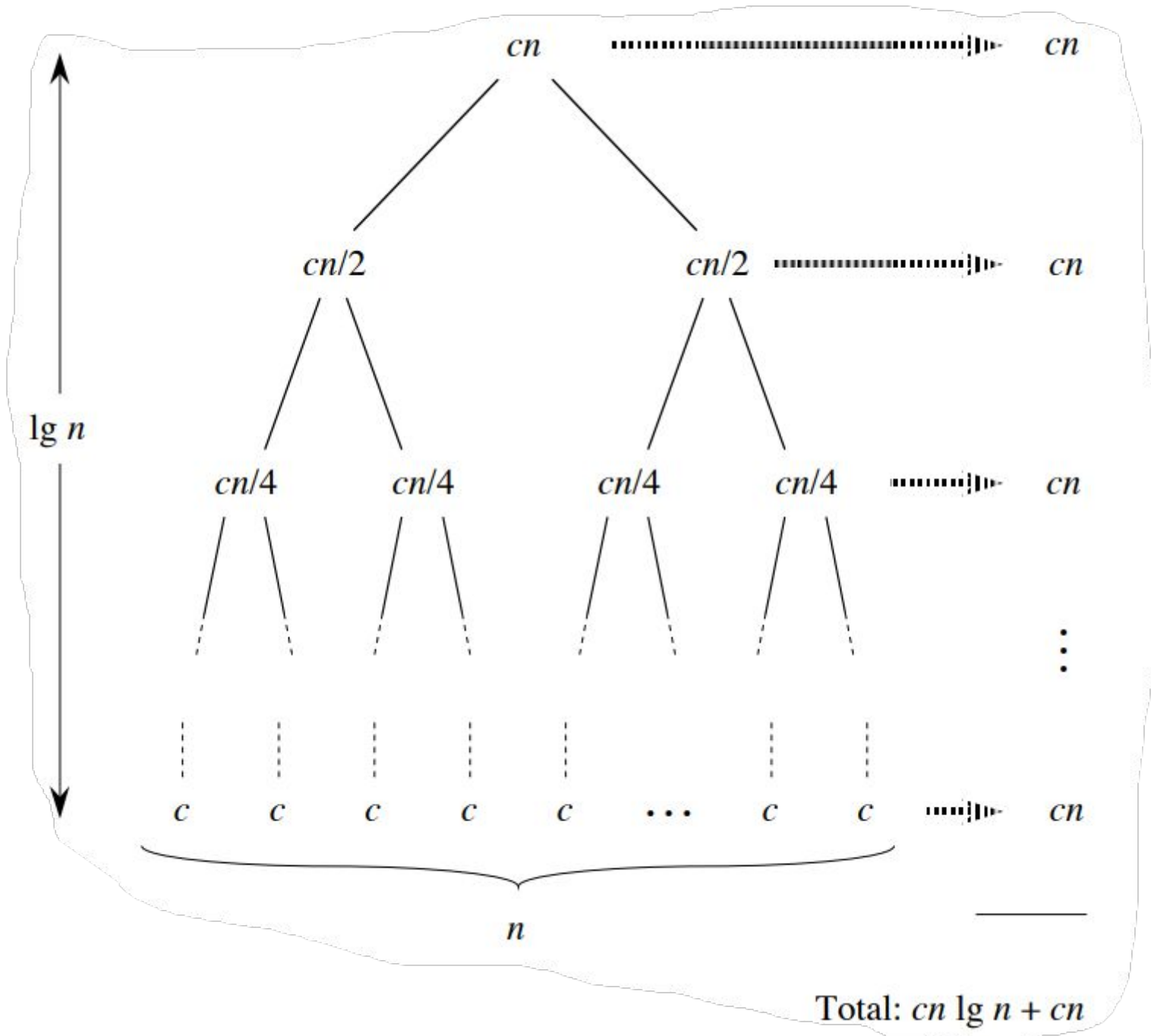
Since  $\Theta(1)$  and  $\Theta(n)$ , summed together they give a function that is linear in  $n$ :  $\Theta(n) \Rightarrow$  recurrence for merge sort running time is

$$T(n) = \begin{cases} \Theta(1) & \text{if, } n = 1 \\ 2T(n/2) + \Theta(n) & \text{if, } n > 1 \end{cases}$$

**Solving this recurrence:**

Rewriting recurrence as:  $T(n) = \begin{cases} c & \text{if } n = 1, \\ 2T(n/2) + cn & \text{if } n > 1. \end{cases}$





**Total cost** is sum of costs at each level. Have  $\log n + 1$  levels, each costing  $cn$

$\Rightarrow$  Total cost is  $T(n) = cn \log n + cn$ .

Ignoring low-order term of  $cn$  and constant coefficient  $c$

$\Rightarrow T(n) = \Theta(n \log n)$ .

**Space Complexity**  $S(n) = O(n)$

two extra arrays are used during the execution

## Quick Sort

Quicksort, like merge sort, is based on the **divide-and-conquer paradigm**. Here is the three-step divide and-conquer process for sorting a typical subarray  $A[p \dots r]$ .

**Divide:** Partition the array  $A[p \dots r]$  into two (possibly empty) subarrays  $A[p \dots q - 1]$  and  $A[q + 1 \dots r]$  such that each element of  $A[p \dots q - 1]$  is  $\leq A[q]$ , and  $A[q] \leq$  to each element of  $A[q + 1 \dots r]$ .

Compute the index  $q$  as part of this partitioning procedure.

**Conquer:** Sort the two subarrays  $A[p \dots q - 1]$  and  $A[q + 1 \dots r]$  by recursive calls to quicksort.

**Combine:** Since the subarrays are sorted in place, no work is needed to combine them: the entire array  $A[p \dots r]$  is now sorted.



**QUICKSORT**(A, p,r)

**if**  $p < r$

**then**  $q \leftarrow \text{PARTITION}(A, p,r)$

**QUICKSORT**(A, p,  $q - 1$ )

**QUICKSORT**(A,  $q + 1,r$ )

Initial call is **QUICKSORT**(A, 0, n-1)

**Partitioning:** Perform the divide step by a procedure **PARTITION**, which returns the index  $q$  (pivot) that marks the position separating the subarrays. Partition subarray  $A[p \dots r]$  by the following procedure:

**PARTITION**(A,  $p, r$ )

$x \leftarrow A[r]$

$i \leftarrow p - 1$

**for**  $j \leftarrow p$  **to**  $r - 1$

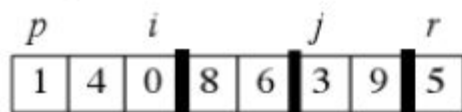
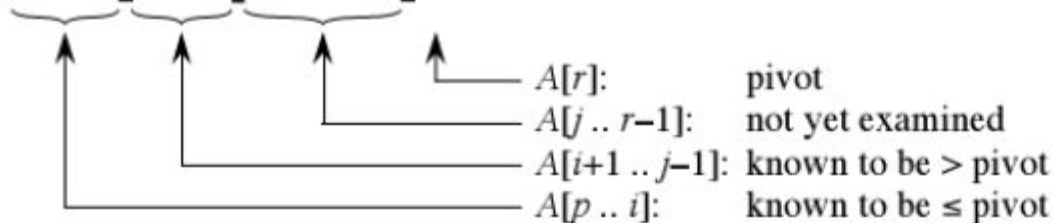
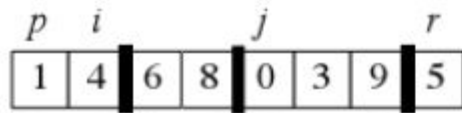
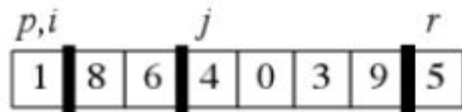
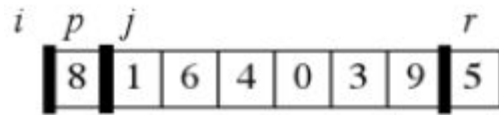
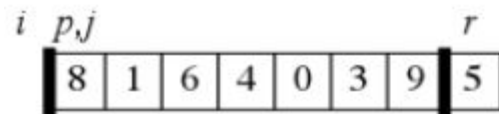
**do if**  $A[j] \leq x$

**then**  $i \leftarrow i + 1$

        exchange  $A[i] \leftrightarrow A[j]$

  exchange  $A[i + 1] \leftrightarrow A[r]$

**return**  $i + 1$



PARTITION( $A, p, r$ )

$x \leftarrow A[r]$

$i \leftarrow p - 1$

**for**  $j \leftarrow p$  **to**  $r - 1$

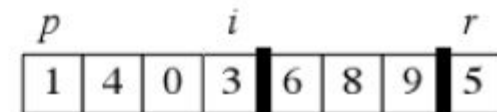
**do if**  $A[j] \leq x$

**then**  $i \leftarrow i + 1$

exchange  $A[i] \leftrightarrow A[j]$

exchange  $A[i + 1] \leftrightarrow A[r]$

**return**  $i + 1$



## Quick sort complexity analysis

The running time of quicksort depends on the partitioning of the subarrays:

- If the subarrays are balanced, then quicksort can run as fast as merge sort.
- If they are unbalanced, then quicksort can run as slowly as insertion sort.

### Worst case

Occurs when the subarrays are completely **unbalanced**.

Have 0 elements in one subarray and  $n - 1$  elements in the other subarray.

**Getting the recurrence:**

$$\begin{aligned} T(n) &= T(n - 1) + T(0) + \Theta(n) \\ &= T(n - 1) + \Theta(n) \\ &= \Theta(n^2) . \end{aligned}$$

## Best case

Occurs when the sub arrays are **completely balanced every time**.

**Each subarray has  $\leq n/2$  elements.**

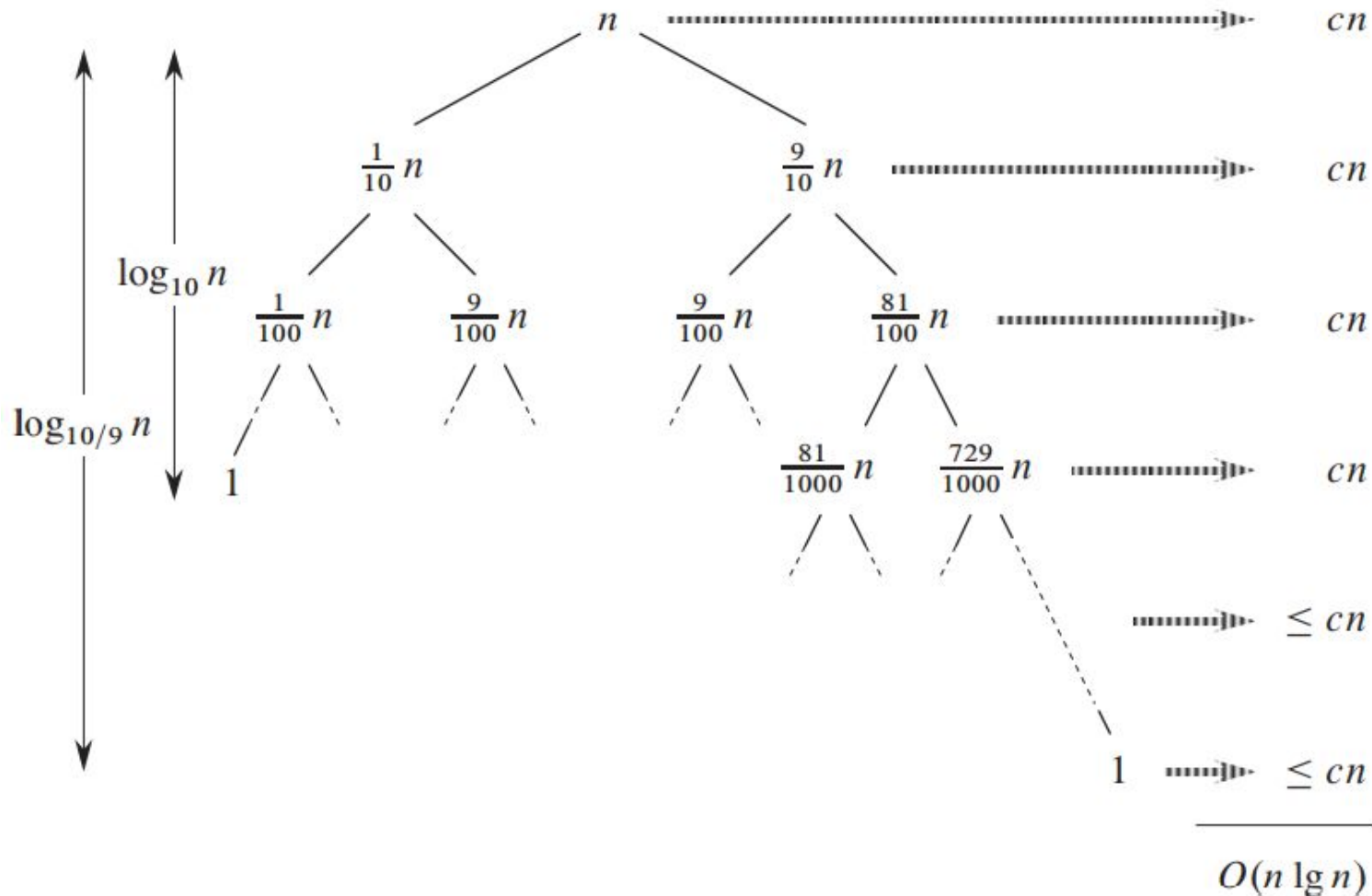
Get the recurrence

$$\begin{aligned} T(n) &= 2T(n/2) + \Theta(n) \\ &= \Theta(n \lg n) . \end{aligned}$$

# Balanced partitioning

Quick sort's average running time is much closer to the best case than to the worst case. Imagine that **PARTITION** always produces a **9-to-1 split**.

Get the recurrence  $T(n) \leq T(9n/10) + T(n/10) + (n) = O(n \lg n)$ .



# Randomized Quick Sort

The algorithm is called randomized if its behavior depends on input as well as random value generated by random number generator. The beauty of the randomized algorithm is that no particular input can produce worst-case behavior of an algorithm.

Randomization is added to quick sort by **not always using  $A[r]$  as the *pivot***, but instead randomly picking (*random sampling*) an element from the subarray that is being sorted.

RANDOMIZED-PARTITION( $A, p, r$ )

$i = \text{RANDOM}(p, r)$

exchange  $A[r]$  with  $A[i]$

**return** PARTITION( $A, p, r$ )

RANDOMIZED-QUICKSORT( $A, p, r$ )

**if**  $p < r$

$q = \text{RANDOMIZED-PARTITION}(A, p, r)$

RANDOMIZED-QUICKSORT( $A, p, q - 1$ )

RANDOMIZED-QUICKSORT( $A, q + 1, r$ )

## Complexity Analysis

### Worst-case

We will prove that a worst-case split at every level produces a worst-case running time of  $O(n^2)$ .

Recurrence for the worst-case running time of QUICKSORT:

Because PARTITION produces two sub problems, totaling size  $n - 1$ ,  $q$  ranges from 0 to  $n - 1$ .

$$T(n) = \max_{0 \leq q \leq n-1} (T(q) + T(n-q-1)) + \Theta(n)$$

Using substitution method to solve this recurrence:

**Guess:**  $T(n) \leq cn^2$ , for some  $c$ .

Substituting our guess into the above recurrence:



**Guess:**  $T(n) \leq cn^2$ , for some  $c$ .

Substituting our guess into the above recurrence:

$$\begin{aligned} T(n) &\leq \max_{0 \leq q \leq n-1} (cq^2 + c(n-q-1)^2) + \Theta(n) \\ &= c \cdot \max_{0 \leq q \leq n-1} (q^2 + (n-q-1)^2) + \Theta(n) \end{aligned}$$

The maximum value of  $(q^2 + (n - q - 1)^2)$  occurs when  $q$  is either 0 or  $n - 1$ .  
(Second derivative with respect to  $q$  is positive meaning the function is increasing.)

This means that

$$\begin{aligned} \max_{0 \leq q \leq n-1} (q^2 + (n-q-1)^2) &\leq (n-1)^2 \\ &= n^2 - 2n + 1 \end{aligned}$$

$$\begin{aligned} \text{Therefore, } T(n) &\leq cn^2 - c(2n - 1) + \Theta(n) \\ &\leq cn^2 \quad \text{if } c(2n - 1) \geq \Theta(n). \end{aligned}$$

Pick  $c$  so that  $c(2n - 1)$  dominates  $\Theta(n)$ .

**Therefore, the worst-case running time of quicksort is  $O(n^2)$ .**

# Average case

To analyze average case, assume that all the input elements are distinct for simplicity. If we are to take care of duplicate elements also the complexity bound is same but it needs more intricate analysis.

**Consider the probability of choosing pivot from  $n$  elements is equally likely i.e.  $1/n$ .**

$$T(n) = 1/n \sum_{k=1}^{n-1} (T(k) + T(n-k)) + O(n).$$

$T(k)$  and  $T(n-k)$  are repeated two times i.e there are 2  $T(1)$ 's ,  
2  $T(2)$ 's ... 2  $T(n-1)$ 's while running through  $1 \leq k \leq n-1$

$$T(n) = 2/n \sum_{k=1}^{n-1} T(k) + O(n)$$

$$nT(n) = 2 \sum_{k=1}^{n-1} T(k) + O(n^2)$$

**Similarly**

$$(n-1)T(n-1) = 2 \sum_{k=1}^{n-2} T(k) + O(n-1)^2$$

**Now,**

$$nT(n) - (n-1)T(n-1) = 2T(n-1) + 2n-1$$

$$nT(n) - (n+1)T(n-1) = 2n-1$$

**Dividing both sides by  $n(n+1)$**

$$nT(n) - (n+1)T(n-1) = 2n-1$$

Dividing both sides by  $n(n+1)$

$$\frac{T(n)}{n+1} = \frac{T(n-1)}{n} + \frac{(2n-1)}{n(n+1)}$$

Let  $A_n = \frac{T(n)}{n+1}$  then

$$A_n = A_{n-1} + \frac{(2n-1)}{n(n+1)}$$

$$A_n = \sum_{i=1}^n \frac{2i-1}{i(i+1)} \leq \frac{2i}{i(i+1)} = 2 \sum_{i=1}^n \frac{1}{(i+1)} \leq 2 \log n \quad // \text{ Sum of Harmonic Series}$$

Substituting the value of  $A_n$

$$\frac{T(n)}{n+1} \leq 2 \log n$$

$$T(n) \leq 2(n+1) \log n = 2n \log n + 2 \log n$$

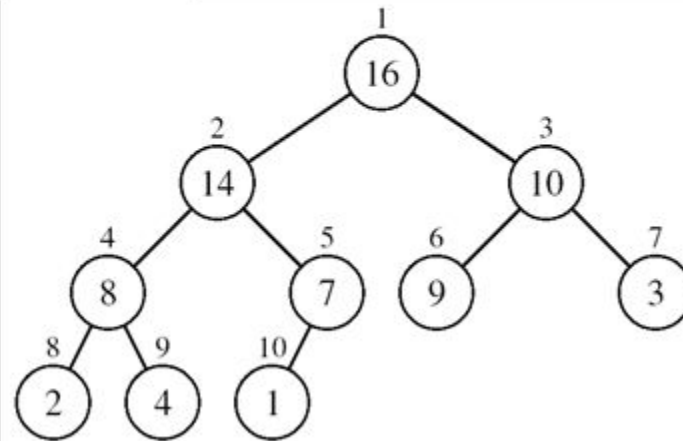
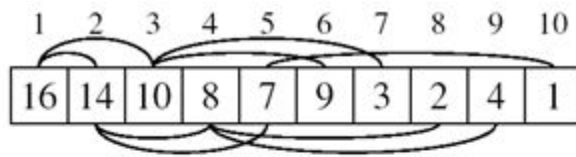
$$T(n) = O(n \log n).$$

# Heap Sort

NEXT

# Array Representation of Heaps

- A heap can be stored as an array  $A$ .
  - Root of tree is  $A[1]$
  - Left child of  $A[i] = A[2i]$
  - Right child of  $A[i] = A[2i + 1]$
  - Parent of  $A[i] = A[\lfloor i/2 \rfloor]$
  - $\text{Heapsize}[A] \leq \text{length}[A]$
- The elements in the subarray  $A[(\lfloor n/2 \rfloor + 1) \text{ to } n]$  are leaves



# Heaps Sort

## Operations on Heaps

- Maintain/Restore the max-heap property
  - **MAX-HEAPIFY**
- Create a max-heap from an unordered array
  - **BUILD-MAX-HEAP**
- Sort an array in place
  - **HEAPSORT**
- Priority queues/ root element always highest/lowest priority

Max-Heapify(A, i, n)

```
{
    l = Left(i)
    r = Right(i)
    largest=i;
    if l ≤ n and A[l] > A[largest]
        largest = l
    if r ≤ n and A[r] > A[largest]
        largest = r
    if largest ≠ i
        exchange (A[i] , A[largest])
        Max-Heapify(A, largest, n)
}
```

## Analysis:

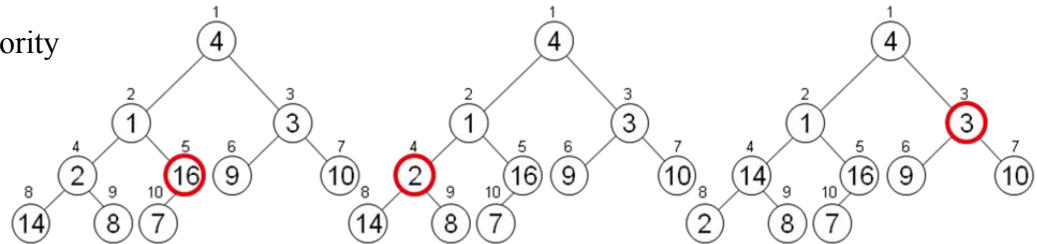
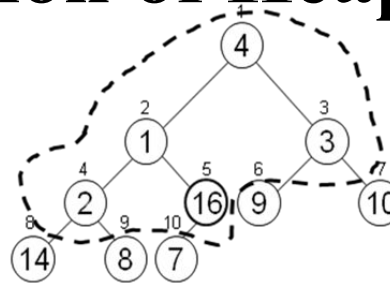
In the worst case Max-Heapify is called recursively  $h$  times, where  $h$  is height of the heap and since each call to the heapify takes constant time

Time complexity =  $O(h) = O(\log n)$

# Array Representation of Heaps

## Operations on Heaps

- Maintain/Restore the max-heap property
  - MAX-HEAPIFY
- Create a max-heap from an unordered array
  - **BUILD-MAX-HEAP**
- Sort an array in place
  - HEAPSORT
- Priority queues/ root element always highest/lowest priority



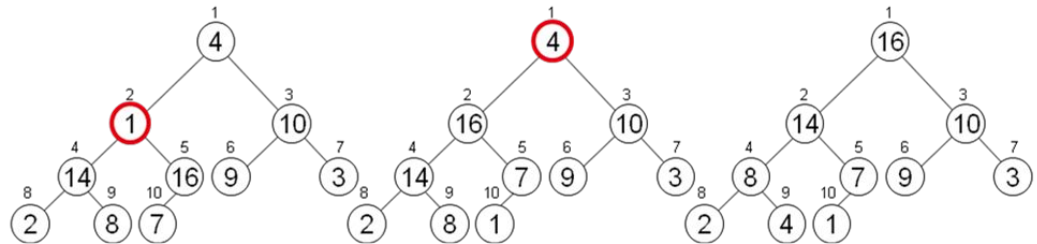
## Algorithm:

Build-Max-Heap(A)

$n = \text{length}[A]$

**for**  $i \leftarrow \lfloor n/2 \rfloor$  **downto** 1

**do** MAX-HEAPIFY(A, i, n) //  $\log n$  times



## Time Complexity:

Running time: Loop executes  $O(n)$  times and complexity of Heapify is  $O(\lg n)$ , therefore complexity of Build-Max-Heap is  $O(n \lg n)$ .



# Asymptotically tight upper bound for max heapify

Heapify takes  $O(h)$

$\Rightarrow$  The cost of Heapify on a node  $i$  is proportional to the height of the node  $i$  in the tree

$$\Rightarrow T(n) = \sum_{i=0}^h n_i h_i$$

$h_i = h - i$  height of the heap rooted at level  $i$

$n_i = 2^i$  number of nodes at level  $i$

$$\Rightarrow T(n) = \sum_{i=0}^h 2^i (h-i)$$

$$\Rightarrow T(n) = \sum_{i=0}^h 2^h (h-i) / 2^{h-i}$$

Let  $k = h-i$

$$\Rightarrow T(n) = 2^h \sum_{i=0}^h k / 2^k$$

$$\Rightarrow T(n) \leq n \sum_{i=0}^{\infty} k / 2^k$$

We know that,  $\sum_{i=0}^{\infty} x^k = 1/(1-x)$  for  $|x| < 1$

Differentiating both sides we get,

$$\sum_{i=0}^{\infty} k x^{k-1} = 1/(1-x)^2$$

$$\sum_{i=0}^{\infty} k x^k = x/(1-x)^2$$

Put  $x=1/2$

$$\sum_{i=0}^{\infty} k / 2^k = 1/(1-x)^2 = 2$$

$$\Rightarrow T(n) = O(n)$$

# Heap Sort

HeapSort(A)

```
{  
    Build-Max-Heap(A); //into max heap  
    n = length[A];  
    for(i = n ; i >= 2; i--)  
    {  
        swap(A[1],A[n]);  
        n = n-1;  
        Max-Heapify(A,1);  
    }  
}
```

example

# Order statistic

- $i^{\text{th}}$  order statistic of a set of elements gives  $i^{\text{th}}$  largest(smallest) element.  
Then minimum is first order statistic and the maximum is last order statistic.  
This kind of problem commonly called selection problem.
- This problem can be solved in  $\Theta(n \log n)$  in a very straightforward way.

## Nonlinear general selection algorithm// Brute force Approach

Find the most extreme value and move it to the beginning until we reach our desired index

```
Select(A, k)
{
  for( i=0;i<k;i++)
  {
    minindex = i;
    minvalue = A[i];
    for(j=i+1;j<n;j++)
    {
      if( A[j] < minvalue)
      {
        minindex
        = j;
        minvalue = A[j];
      }
    }
    swap(A[i],A[minIndex]);
  }
  return A[k];
}
```

# Selection in expected linear time

The idea for this problem solving is to partition the element set as in Quick Sort where partition is randomized one.

```
RandSelect(A,p,r,i)
{
    if(p = r )
        return A[p];
    q = RandPartition(A,p,r);
    k = q - p + 1;
    if i=k //pivot is answer
        then return A[q]
    if(i < k)
        return RandSelect(A,p,q-1,i);
    else
        return RandSelect(A,q+1,r,i - k);
}
```

# Worst-case linear-time order statistics

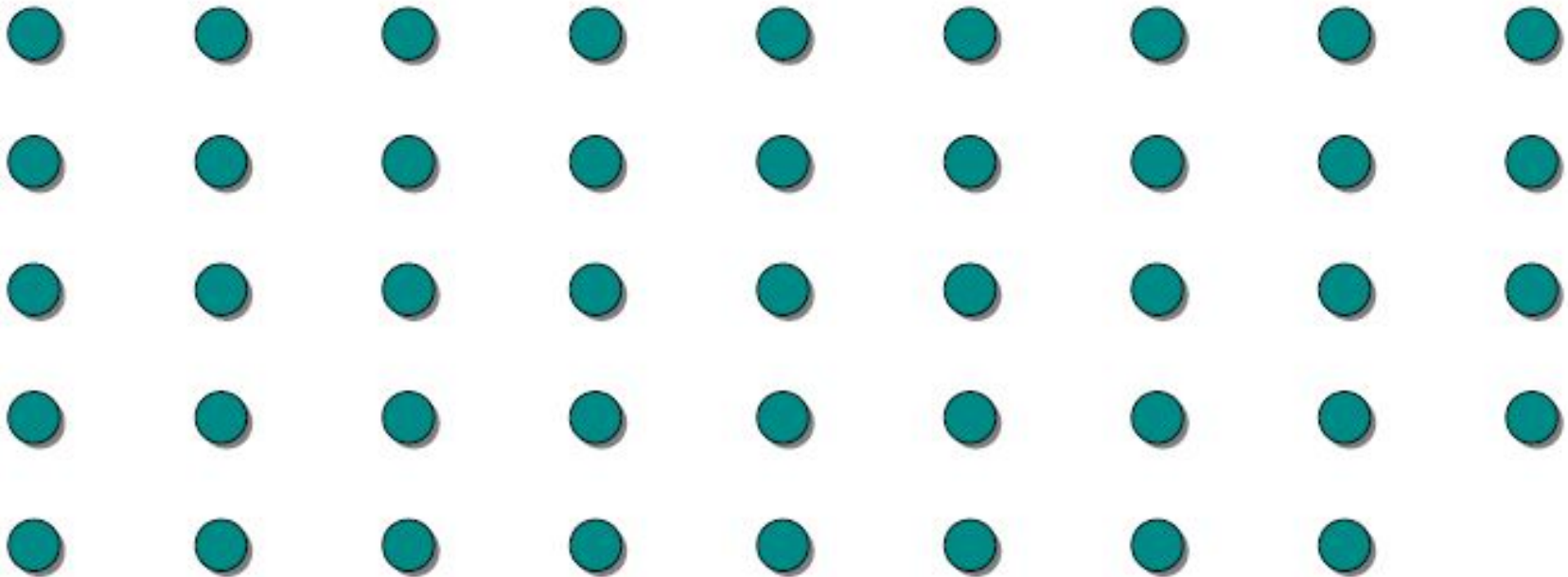
- Divide the  $n$  elements into groups of 5.
- Find the median of each 5-element group.
- Recursively SELECT the median  $x$  of the  $\lfloor n/5 \rfloor$  group medians to be the pivot.

## Algorithm:

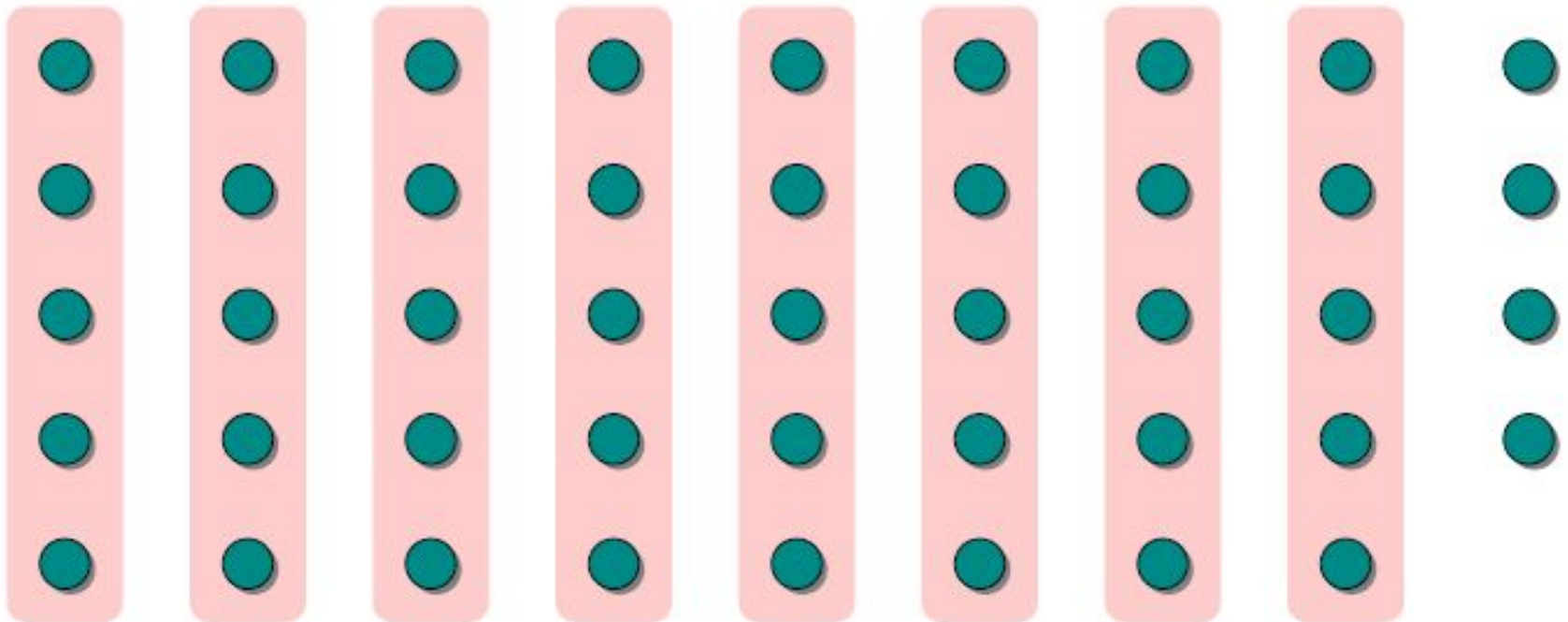
### Select( $l, n$ )

1. Divide  $n$  elements into groups of 5  
Find median of each group
2. Use Select() recursively to find median  $x$  of the  $\lfloor n/5 \rfloor$  medians
3. Partition the  $n$  elements around  $x$ . Let  $k = \text{rank}(x)$  // index of  $x$
4. **if** ( $i == k$ ) **then** return  $x$   
**else if** ( $i < k$ ) **then**  
 use Select() recursively to find  $i$ th smallest element in first partition  
**else** ( $i > k$ ) use Select() recursively to find  $(i-k)$ th smallest element in upper partition.

# Choosing the pivot



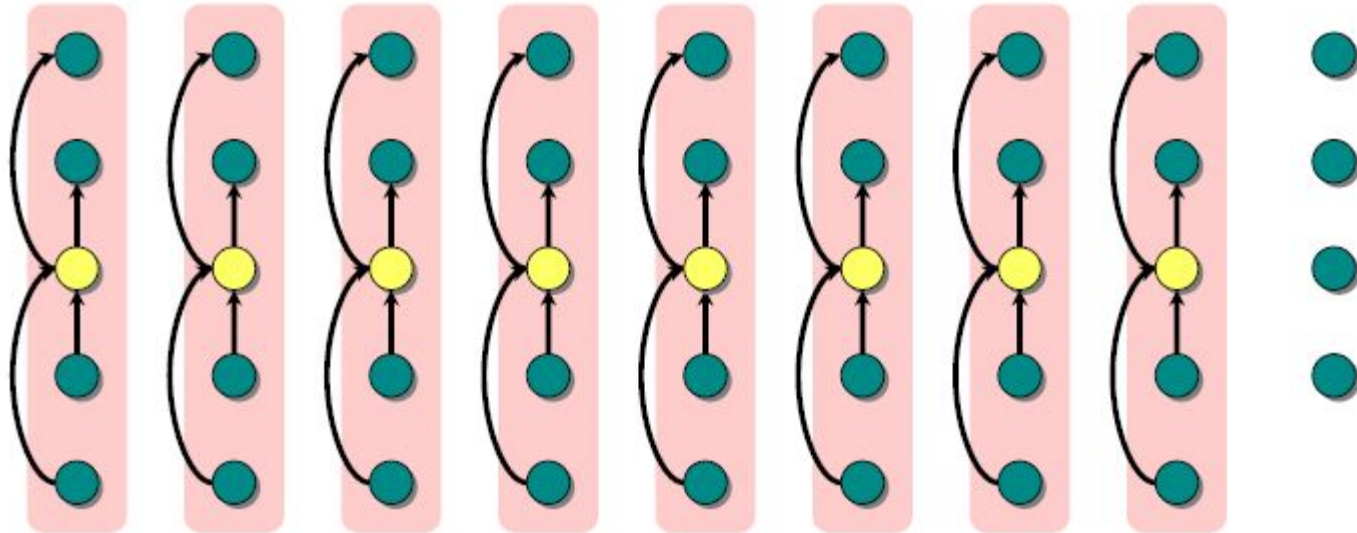
# Choosing the pivot



1. Divide the  $n$  elements into groups of 5.

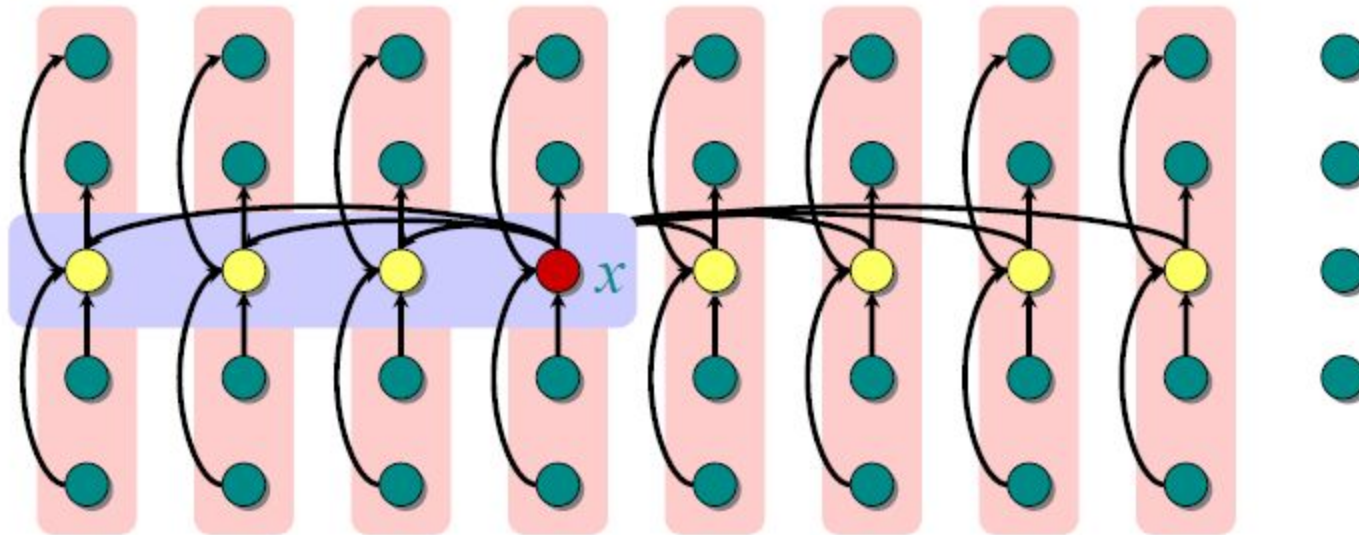


# Choosing the pivot



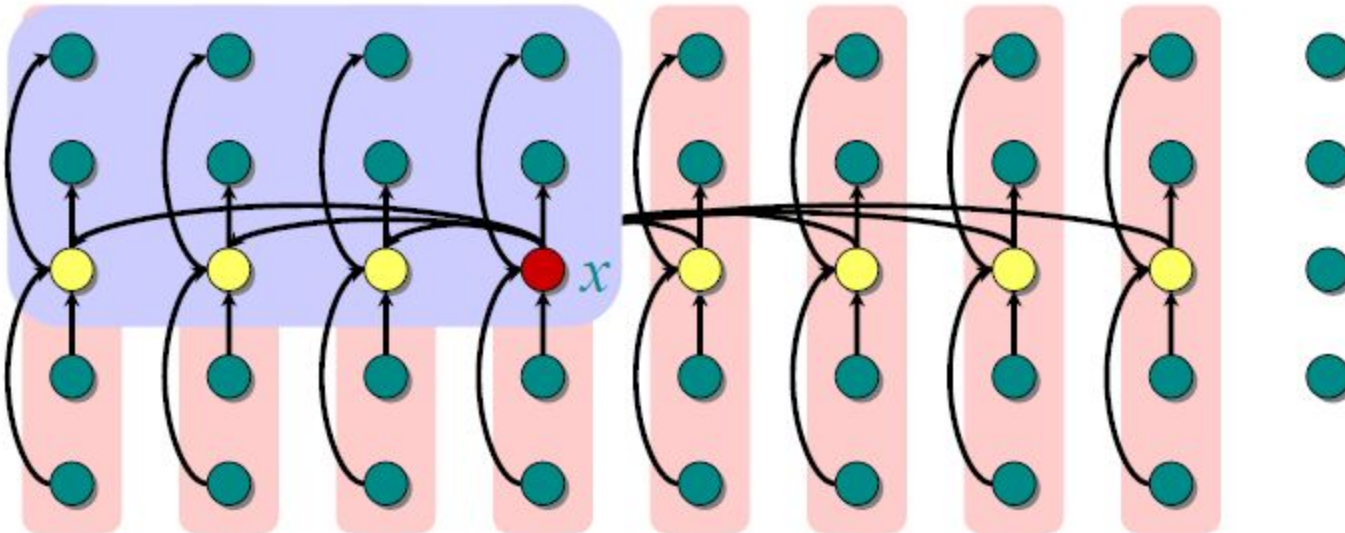
1. Divide the  $n$  elements into groups of 5. Find the median of each 5-element group by rote.
2. Recursively SELECT the median  $x$  of the  $\lfloor n/5 \rfloor$  group medians to be the pivot.

# Analysis

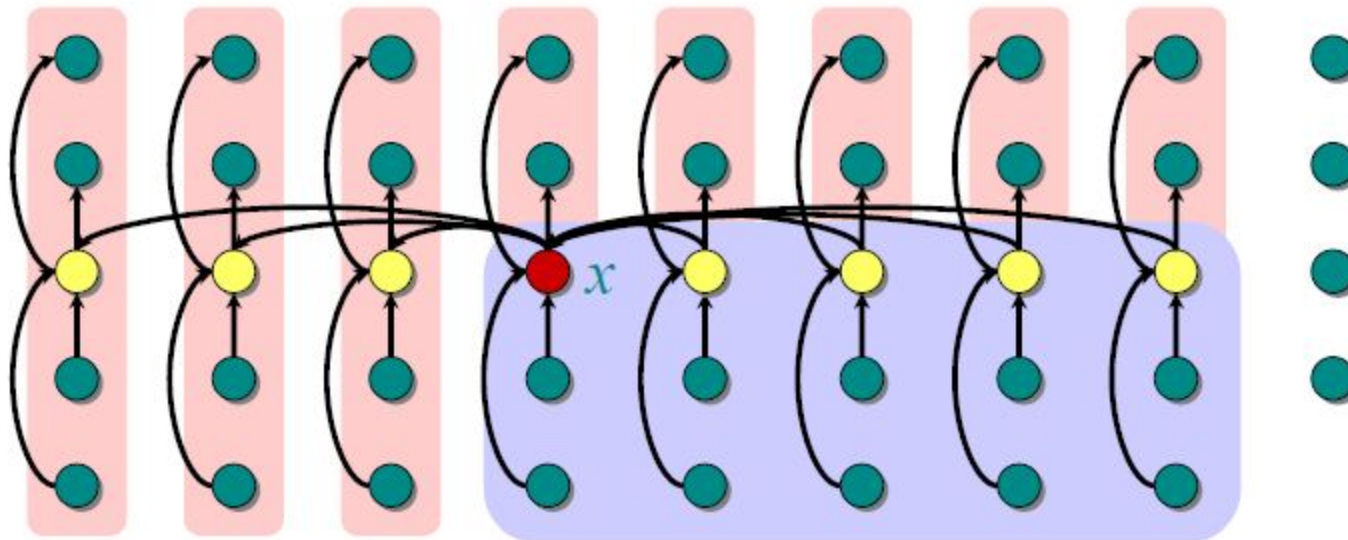


At least half the group medians are  $\leq x$ ,  
 which is at least  $\lfloor \lfloor n/5 \rfloor / 2 \rfloor = \lfloor n/10 \rfloor$  group medians.

# Analysis



At least half the group medians are  $\leq x$ , which is at least  $\lfloor \lfloor n/5 \rfloor / 2 \rfloor = \lfloor n/10 \rfloor$  group medians. Therefore, at least  $3\lfloor n/10 \rfloor$  elements are  $\leq x$ .



At least half the group medians are  $\leq x$ , which is at least  $\lfloor \lfloor n/5 \rfloor / 2 \rfloor = \lfloor n/10 \rfloor$  group medians.  
 Therefore, at least  $3\lfloor n/10 \rfloor$  elements are  $\leq x$ .  
 Similarly, at least  $3\lfloor n/10 \rfloor$  elements are  $\geq x$ .

# Simplification

Remaining elements

$$n - 3n/10 = 7n/10$$

Now

$$T(n) = T(n/5) + T(7n/10) + O(n)$$

Can be solved by substitution for linear time

$$\text{i.e } T(n) = O(n).$$

# Developing the recurrence

|             |                                                                                                                                                                                                                                          |
|-------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| $T(n)$      | SELECT( $i, n$ )                                                                                                                                                                                                                         |
| $\Theta(n)$ | 1. Divide the $n$ elements into groups of 5. Find the median of each 5-element group by rote.                                                                                                                                            |
| $T(n/5)$    | 2. Recursively SELECT the median $x$ of the $\lfloor n/5 \rfloor$ group medians to be the pivot.                                                                                                                                         |
| $\Theta(n)$ | 3. Partition around the pivot $x$ . Let $k = \text{rank}(x)$ .                                                                                                                                                                           |
| $T(3n/4)$   | 4. <b>if</b> $i = k$ <b>then return</b> $x$<br><b>elseif</b> $i < k$<br><b>then</b> recursively SELECT the $i$ th smallest element in the lower part<br><b>else</b> recursively SELECT the $(i-k)$ th smallest element in the upper part |



# Solving the recurrence

$$T(n) = T\left(\frac{1}{5}n\right) + T\left(\frac{3}{4}n\right) + \Theta(n)$$

---

**Substitution:**

$$T(n) \leq cn$$

$$\begin{aligned} T(n) &\leq \frac{1}{5}cn + \frac{3}{4}cn + \Theta(n) \\ &= \frac{19}{20}cn + \Theta(n) \\ &= cn - \left(\frac{1}{20}cn - \Theta(n)\right) \\ &\leq cn, \end{aligned}$$

if  $c$  is chosen large enough to handle both the  $\Theta(n)$  and the initial conditions.